



HashCloak

Code Review and Security Assessment
For
Sunspot

Initial Report: May 22nd, 2026

Updated Review: May 28th, 2026

Updated Version 2.0: Sept 23rd, 2026

Prepared For:

Matthew Klein | Reilabs

Marcin Kostrzewa | Reilabs

Cat McGee | Solana Foundation

Prepared by:

Alex Yu-Chen Liao | *HashCloak Inc.*

Manish Kumar | *HashCloak Inc.*

Table Of Contents

Executive Summary	2
Scope	3
Overview	3
Methodology	3
Overview of Evaluated Components	4
Findings	6
SNT-1: Only 16 of 32 Output Bytes Constrained for blake2 hash	6
SNT-2: The high limb portion of a scalar is ignored in MultiScalarMul	7
SNT-3: No point validation for grumpkin curve in embedded_curve_add and multi_scalar_mul	8
SNT-4: No point validation for secp256k1/secp256r1 point in ECDSA	9
SNT-5: No BN254 subgroup verification for Groth16 proof and VK in recursive_aggregation.	11
SNT-6: And and Xor use uints.U64 regardless of nBits; computations larger than 64 bits fails	12
SNT-7: Secp256k1 ECDSA signature (r, s) with s > order/2 is not allowed in Noir but would pass in Sunspot	14
SNT-8: KeyHash in the recursive proof is an unconstrained free witness	14
SNT-9: Witness index in FillWitnessTree() might not match the number of witnesses in the tree for a subCircuit	16
SNT-10: Define() in circuit.go only collects private inputs	18
SNT-11: MultiScalarMul.Define calls grumpkin.MultiScalarMul without algopts.WithCompleteArithmetic(); performs unsafe arithmetics for scalar multiplication	20
SNT-12: Commitment Loop Never Advances index idx while creating new proof in recursive aggregation	21
SNT-13: is_infinite flag for points are ignored	21
SNT-14: Program.UnmarshalReader reads unconstrainedFuncCount with the wrong endianness and ignores reading any UnconstrainedFunctions	22
SNT-15: Mismatch behavior for multi_scalar_mul black box function when scalar >= Fr	24
SNT-16: Incorrect Unmarshalling for BrilligBytecode	24
SNT-17: Unsafe operations in ECC arithmetic	26
SNT-18: Blackbox function range in Sunspot does not consider constant witness input	27
SNT-19: Missing Equality implementation for BN254Field	28
SNT-20: Non-EOF Errors Silently ignored When Reading numAssertMessages	29
SNT-21: errors discarded silently	30
SNT-22: Wrong logical operator in guard condition while decomposing Scalar	31
SNT-23: Use IsWitness() instead of FunctionInputKind == 1 to check whether a function input is a witness.	32
SNT-24: Use safe type assertion in Equals() for ecdsa_secp256k1.go	33
SNT-25: constantWitnessID declared in Expression is dead code	34
SNT-26: Test cases does not handle all cases in testing	34
SNT-27: out-of-bounds dynamic-array write silently becomes a no-op	36
SNT-28: pub-declared Noir return values are silently compiled as Secret witnesses	37
References	39

Executive Summary

The *Solana Foundation* engaged *HashCloak Inc.* to perform a code review and security audit for Reilabs' Sunspot, a Noir proving backend that provides tools to prove and verify Noir circuits on Solana. The security audit was done from April 1st to May 22nd with 2 auditors.

The engagement began with an in-depth review of Sunspot's overall workflow and the interactions between each component during the compiling, proving, and verifying process. We then thoroughly studied the documentations of Sunspot, Noir, and Gnark, and conducted thread modeling to identify potential risk vectors in the system. Afterward, we performed a detailed analysis of the codebase with particular emphasis on whether the implementation would correctly compile a Noir circuit into a Gnark circuit. We also analyzed the correctness of gnark-solana for verifying Gnark circuits on Solana. We also ran X-ray, a static analyzer tool for solana for detecting common security vulnerabilities in solana.

Our review found that overall the code is well defined and consistency holds across Noir and Gnark circuits. We suggest compiling a complete documentation for the Sunspot proving system to help users understand the overall system design and improve future maintainability.

On May 28, we reviewed the main branch at the following commit to ensure that no new issues have been introduced into the codebase. All issues found in the audit have been resolved or acknowledged as of commit [a58077e58366e0b2750b032652f2588a6776abb4](https://github.com/reilabs/sunspot/commit/a58077e58366e0b2750b032652f2588a6776abb4).

Later, as a part of an internal research project for finding bugs using AI autoresearch patterns, we provided Sunspot codebase for analysis. The research identified 2 high severity issues (SNT-27, SNT-28) which we shared with the Sunspot team. The team addressed the issues and the fixes in the PR <https://github.com/reilabs/sunspot/pull/57>, resolving all the reported vulnerabilities.

Overall, we found the issues range from Critical to Informational:

Severity	Number of Findings	Status
Critical	3	Resolved
High	13	Resolved
Medium	3	Resolved
Low	4	Resolved
Informational	5	Resolved

Scope

- <https://github.com/reilabs/Sunspot> at commit 3742263b4bd2be13a61f17d146c7545527529e55 on main branch

Overview

Sunspot is a Noir proving backend that provides tools to prove and verify Noir circuits on Solana. It includes a translation layer that will first compile a Noir circuit into a Gnark circuit. The compiler supports general Noir expressions as well as all the blackbox functions. Afterward, it can output proofs in Gnark, which uses an enhanced version of Groth16. Finally, the toolchain then auto-generates a Solana verifier program using the gnark-solana library, enabling users to verify proofs on Solana. The verification would pass only if the witness conforms to the original Noir circuit, essentially creating an efficient way for users to verify a Noir circuit on Solana.

Methodology

The audit was conducted through a combination of thread modeling and manual code review. Based on the security requirements of Sunspot, we developed several security properties that each component must follow, especially for the correctness and soundness when translating a Noir circuit to Gnark. During the manual review phase, we first developed a thorough understanding of the overall workflow in Sunspot and how it is done in the Noir compiler to make sure both circuits would match. We then systematically assessed the codebase against identified areas of concern and potential attack surfaces during the thread modeling.

Overview of Evaluated Components

Gnark-solana

- Correctness of hash to field
- Witness parsing

- Correctness of batched pederson commitment verification
- Input validation
- Fiat-shamir challenge generation
- Pairing verification equation
- Syscall Wrappers

Acir

- Blackbox functions matches the implementation in Noir
- Correctness of the blackbox functions
- No underconstraint or overconstraint issues in the blackbox functions
- Correct implementation for Noir opcodes.
- Correct translation of Noir circuit into Gnark
- Correct and sound compilation of Noir circuits
- Underconstraint or overconstraint issues
- Correctly handles the witness.

Bn254

- Correct usage of the gnark-crypto library
- Safety while unmarshaling into field

Cmd

- Loading and saving for the circuit, witness and proof files
- Process of building the SBF files
- Correct proving process
- Correct verifying process

Poseidon2

- Round constant correctness
- Full Round matrix
- Internal matrix multiplication
- State initialization

Sw-grumpkin

- Correctness of point addition, doubling and scalar multiplication
- Handling of edge cases such as point at infinity, $p = \pm q$ and 0 Scalar
- Use of `cfg.CompleteArithmetic`
- Const and variable scalar multiplication
- Scalar decomposition

Findings

SNT-1: Only 16 of 32 Output Bytes Constrained for blake2 hash

Type: Critical

Files affected: go/acir/black_box_func/blake2s.go

Description: The `Blake2Permute` function returns `h[0:4]` which is only the first four 32-bit state words (16 bytes) out of 8 state words. The `Blake2Permute` function is called in `Blake2s.Define` which gives the constrained output as:

```
constrained_output, err := Blake2Permute(api, uapi,  
SplitIntoBlocks16(uapi, data), len(a.Inputs))
```

Followed by the outer loop which adds the equal assertion constraint iterating for `i := range 4` covering only `a.Outputs[0..15]` (the first 16 bytes). The remaining 16 output bytes (`a.Outputs[16..31]`) are registered in `FillWitnessTree`, but no constraint ever ties them to the computed hash. They are completely free variables. That essentially means whatever the prover sets those 16 bytes to can still pass the verification.

Impact: Underconstrained circuit for Blake2s allows a malicious prover to generate a valid proof for a program without full knowledge of the correct hash output, leading to a soundness issue.

Recommendation: Return all 8 state words from `Blake2Permute` (`h[0:8]` instead of `h[0:4]`) and change the loop bound from range 4 to range 8 so that it constrains the entire `a.Outputs`.

Status: The issue has been resolved in commit [2306145294e2804ad406f034a1b7b1072e3b4786](https://github.com/ethereum/go-ethereum/commit/2306145294e2804ad406f034a1b7b1072e3b4786)

SNT-2: The high limb portion of a scalar is ignored in **MultiScalarMul**

Type: Critical

Files affected: go/acir/black_box_func/multi_scalar_mul.go

Description: In the **Define** function of **MultiScalarMul**, it will only take the low limb part of a scalar and discard the high limb part. This is an implementation error, since any scalar with a nonzero high limb would compute a different result than Noir.

Below is an example Prover.toml that will pass in Noir but failed in Sunspot

```
[output]
is_infinite = false
x =
"0x169b0cc066d51f61371af8476004cc5301d935ccad997d7b2228b396793aee5b"
y=
"0x0742cc7bda90efb8453c43b06f8ec7785d3b65c8e5c0930471327dd50c84d123"
[[points]]
is_infinite = false
x =
"13457461695866919433821564521619285514105166235431728247720640848441
866812394"
y =
"12239832528093754184015030410952400611363246321141890794076152105619
117698331"
[[points]]
is_infinite = false
x =
"53413198552995672839625483766428766055611491910240640976510195415030
13001492"
y =
"89471505512786875219853735225240559606583826619411694337736590173097
56569695"
[[points]]
is_infinite = false
x =
```

```
"21190260147211659516065528144212862696681124931670934505230036935494
43806567"
y =
"12520261813199548858744432192769383837819987256498135451689313779647
572966656"
[[scalars]]
hi = 3
lo = 2
[[scalars]]
hi = 2
lo = 6
[[scalars]]
hi = 0
lo = 5
```

Impact: Any scalar with a nonzero high limb will have incorrect outputs.

Recommendation: Read both the high and low limb part and combine it as one scalar.

Status: The Sunspot team has updated the `Define` function of `MultiScalarMul` to combine both high and low limb as one scalar in commit [1e74f01d4f8b3e55114d7b3cad0603298d0ec9ac](https://github.com/sunspot-labs/sunspot/commit/1e74f01d4f8b3e55114d7b3cad0603298d0ec9ac).

SNT-3: No point validation for grumpkin curve in `embedded_curve_add` and `multi_scalar_mul`

Type: High

Files affected:

- `go/acir/black_box_func/embedded_curve_add.go`
- `go/acir/black_box_func/multi_scalar_mul.go`

Description: In the `Define()` function of `embedded_curve_add` and `multi_scalar_mul` the Grumpkin points are coming from unchecked witnesses. Consider verifying the points before the operation.

Impact: If users can freely pick points from different curves, provers might be able to pick such invalid points that will always pass the verification.

Recommendation: Provide an `AssertIsOnCurve()` function in `sw_grumpkin/g1.go` that will check whether a point is valid and call it after a point is created in the `Define()` function in both files. An example `AssertIsOnCurve()` function is as below:

```
func (p *G1Affine) AssertIsOnCurve(api frontend.API) {
    // In case the p is infinity (0,0)
    selector := api.And(api.IsZero(p.X), api.IsZero(p.Y))

    //  $Y^2 = X^3 - 17$ 
    diff := api.Sub(api.Mul(p.Y, p.Y), api.Mul(p.X, p.X, p.X))
    expected := api.Select(selector, 0, big.NewInt(-17))
    api.AssertIsEqual(diff, expected)
}
```

Status: The Sunspot team has added point validation for Grumpkin curve for both files in commit: [8d875c5276a6a89d2e906813bd3e3b28898f269c](https://github.com/sunspot-labs/sunspot/commit/8d875c5276a6a89d2e906813bd3e3b28898f269c).

SNT-4: No point validation for `secp256k1/secp256r1` point in ECDSA

Type: High

Files affected:

- `go/acir/black_box_func/ecdsa_secp256k1.go`
- `go/acir/black_box_func/ecdsa_secp256r1.go`

Description: The public key in `Define()` for both `secp256k1/secp256r1` are unverified witness inputs. Consider validating the public keys before verifying the signature.

Impact: Provers may be able to pick an invalid point as a public key that will always pass the verification.

Recommendation: Check whether the public keys are on curves like below.

For **secp256k1**:

```
//Inside Define()
curve, err := sw_emulated.New[emulated.Secp256k1Fp,
emulated.Secp256k1Fr](api, sw_emulated.GetSecp256k1Params())
if err != nil {
    return err
}

Q := ecdsa.PublicKey[emulated.Secp256k1Fp, emulated.Secp256k1Fr]{
    X: *primeField.NewElement(qXValue),
    Y: *primeField.NewElement(qYValue),
}

pkpt := sw_emulated.AffinePoint[emulated.Secp256k1Fp](Q)
curve.AssertIsOnCurve(&pkpt)
```

For **secp256r1**:

```
//Inside Define()
curve, err := sw_emulated.New[emulated.P256Fp, emulated.P256Fr](api,
sw_emulated.GetP256Params())
if err != nil {
    return err
}

Q := ecdsa.PublicKey[emulated.P256Fp, emulated.P256Fr]{
    X: *primeField.NewElement(qXValue),
    Y: *primeField.NewElement(qYValue),
}

pkpt := sw_emulated.AffinePoint[emulated.P256Fp](Q)
curve.AssertIsOnCurve(&pkpt)
```

Status: The Sunspot team has added point validation in the `Define` function for both `secp256k1` and `secp256r1` as suggested in commit [e29fd6586f9a9f936ace0d71c103f5a4e9d9db76](https://github.com/sunspot/commit/e29fd6586f9a9f936ace0d71c103f5a4e9d9db76).

SNT-5: No BN254 subgroup verification for Groth16 proof and VK in `recursive_aggregation`.

Type: High

Files affected: `go/acir/black_box_func/recursive_aggregation.go`

Description: The Groth16 `VK` and proof are unverified witness input. Consider adding corresponding `G1`, `G2` subgroup checks for the `VK` and using the built-in `WithSubgroupCheck` for proofs in `AssertProof()`.

Impact: Prover might be able to pick invalid proofs and VKs that will always pass the verification.

Recommendation: For VK, check each `G1`, `G2` value in `newVK()`:

```
// in newVK()
for i := range k {
    k_i, err := newG1(api, vars[14+i*2:14+i*2+2], witnesses)
    if err != nil {
        return vk, err
    }
    e.AssertIsOnG1(&k_i)
    k[i] = k_i
}

e.AssertIsOnG1(&alpha)
e.AssertIsOnG2(&g2Beta)
e.AssertIsOnG2(&g2Gamma)
e.AssertIsOnG2(&g2Delta)

for i := range commitments {
```

```

    g, err := newG2(api, vars[idx:idx+4], witnesses)
    if err != nil {
        return vk, err
    }
    gSigmaNeg, err := newG2(api, vars[idx+4:idx+8], witnesses)
    if err != nil {
        return vk, err
    }
    e.AssertIsOnG2(&g)
    e.AssertIsOnG2(&gSigmaNeg)
    commitments[i].G = g
    commitments[i].GSigmaNeg = gSigmaNeg
    idx += 8
}

```

For the proof, pass the `groth16.WithSubgroupCheck()` option for `AssertProof()` in `AggregateGroth16()`:

```

// in AggregateGroth16()
v.AssertProof(vk, proof, witness, groth16.WithSubgroupCheck())

```

Status: The Sunspot team has added subgroup verification for Groth16 proof and VK in commit [158c5e07f5954ab35e851ea99ea2c2d583e67c1c](https://github.com/sunspot-labs/sunspot/commit/158c5e07f5954ab35e851ea99ea2c2d583e67c1c).

SNT-6: And and Xor use `uints.U64` regardless of `nBits`; computations larger than 64 bits fails

Type: High

Files affected:

- go/acir/black_box_func/and.go
- go/acir/black_box_func/xor.go

Description: Both the function `And.Define` and `Xor.Define` instantiate the gnark uint API as `uints.New[uints.U64](api)`. This limits the working word size to

64bits regardless of `nBits` value. The inputs are correctly range checked to `nBits` via `rangechecker.Check(lhs, int(a.nBits))` but immediately after that `uapi.ValueOf(output)` decomposes the field element into a long integer (`uints.U64`). If the witness size has `nBits` more than 64, that does not fit into the `uints.U64` resulting in proof generation failure at runtime. It should be noted that the max supported integer type in Noir circuit for `And` and `Xor` operations is `u128`(128 bit).

Below given the Noir program and Prover.toml which will pass the Noir proving and verification but crashes during Sunspot proving:

File: `noir-samples/black_box_functions/and/src/main.nr`

```
fn main(x: u128, y: pub u128, z: u128) {  
    assert(x & y == z);  
}
```

File: `noir-samples/black_box_functions/and/Prover.toml`

```
x = "139407991430939255523467833655006660152"  
y = "122322932556009385378279989227083497117"  
z = "95704418549659270790766610092885345816"
```

Impact: Proof generation crashes for inputs with bit size more than 64.

Recommendation: `And.Define` and `Xor.Define` must be taken into account to support 128 bit And and Xor operation which can be implemented by decomposing the `u128` into two `u64` limbs and asserting that the decomposition was done correctly.

Status: The issue has been resolved in commit [e937426ab9d24e410cee0acb344d33253bd5ce0e](https://github.com/noir-lang/noir/pull/1000).

SNT-7: Secp256k1 ECDSA signature (r, s) with $s > \text{order}/2$ is not allowed in Noir but would pass in Sunspot

Type: High

Files affected: go/acir/black_box_func/ecdsa_secp256k1.go

Description: In Noir, the verification of ECDSA & EDDSA signatures doesn't allow $(r, -s)$, as seen in the Noir [documentation](#). But in [gnark](#) It only checks that r and $s \leq \text{scalarApi.Modulus}$.

Impact: Since this condition is required in Noir, users might assume the same condition holds in Sunspot and could lead to a malleability attack where an attacker can derive $(r, -s)$ from a signature (r, s) and pass the verification.

Recommendation: Enforce the low-S check in the `Define()` function like below:

```
sElement := scalarField.NewElement(sValue)
// secp256k1 N/2
halfOrder :=
scalarField.NewElement("578960446186580977117854925043439539264187821
39537452191302581570759080747168")
scalarField.AssertIsLessOrEqual(sElement, halfOrder)
```

Status: The Sunspot team has added the low-S check for `secp256k1` in commit [ed3f64fd8c7c12bc976d359adc2af93509d72](#).

SNT-8: `KeyHash` in the recursive proof is an unconstrained free witness

Type: High

Files affected: go/acir/black_box_func/recursive_aggregation.go

Description: The `KeyHash` is currently unconstrained in the recursive proof and can be set to any value by the prover.

Impact: This issue may lead to malleability attacks where a prover generates multiple valid proofs with different `KeyHash`s. Or if a circuit on Sunspot is only verifying the `KeyHash` instead of the actual verification key, users may be able to generate a fake proof with incorrect `VerificationKey` but correct `KeyHash` and still pass the verification, since this is the intended usage for the `RecursiveAggregationblackbox` function(see the noir code [here](#)).

Recommendation: Consider implementing the `KeyHash` functionality that will recalculate the hash and compare it with the input to prevent malleability and misuse in circuits. See the implementation in the [Barretenberg backend](#).

Below is an implementation example using poseidon2 hash from Gnark:

```
Import{
  gnarkhash "github.com/consensys/gnark/std/hash"
  poseidon2perm
  "github.com/consensys/gnark/std/permutation/poseidon2"
}

func (a *RecursiveAggregation[T, E]) AggregateGroth16(api
frontend.Builder[E], witnesses map[shr.Witness]frontend.Variable)
error {

  ...

  // calculate and check VK_hash
  perm, err := poseidon2perm.NewPoseidon2FromParameters(api, 2, 6,
50)
  if err != nil {
    return err
  }
  hasher := gnarkhash.NewMerkleDamgardHasher(api, perm, 0)

  for i := range a.VerificationKey {
    v, err := a.VerificationKey[i].ToVariable(witnesses)
```

```

        if err != nil {
            return err
        }
        hasher.Write(v)
    }
    computed := hasher.Sum()

    keyHashVar, err := a.KeyHash.ToVariable(witnesses)
    if err != nil {
        return err
    }
    api.AssertIsEqual(computed, keyHashVar)

    ...
}

```

Status: The Sunspot team has constrained the `KeyHash` to 0 to prevent the malleability and fake VK issue in commit [718e3599c6f79aa05db30fb43a819dd947ef91fc](https://github.com/sunspot-labs/sunspot/pull/118).

SNT-9: Witness index in `FillWitnessTree()` might not match the number of witnesses in the tree for a `subCircuit`

Type: High

Files affected: go/acir/circuit.go

Description: In the function `FillWitnessTree`, the current witness index is increased by counting the witness inside the `subCircuitWitnessTree`. However, if there's an unused witness in the sub-circuit, the number of witnesses in the `subCircuitWitnessTree` won't include the unused witness and thus the index will be undercounted by 1.

The following circuit will panic while proving:

```

fn main(x: Field, y: pub Field, a: pub u64, b: u64) {
    let z1 = foo(a, y, b);
    let z2 = x + z1 as Field;
    assert(z2 == 5);
}

#[fold]
fn foo(x: u64, z: Field, y: u64) -> u64 {
    let r = x & y;
    r
}

```

Impact: Since `z` in `foo()` is unused, witness `z` won't be included in `subCircuitWitnessTree()`, and the index will be undercounted.

Recommendation: Consider adding the following lines at the end of `FillWitnessTree()` in `circuit.go`

```

func (c *Circuit[T, E]) FillWitnessTree(witnessTree *btree.BTree,
    resolve CircuitResolver[T, E], index uint32) (uint32, error) {

    ...

    // insert witness in the parameter trees in case it is unused in
    any opcode
    c.PrivateParameters.Ascend(func(it btree.Item) bool {
        witness, ok := it.(shr.Witness)
        if !ok {
            panic("Item in private parameter tree not of type witness")
        }
        witnessTree.ReplaceOrInsert(shr.Witness(index) + witness)
        return true
    })
    c.PublicParameters.Ascend(func(it btree.Item) bool {
        witness, ok := it.(shr.Witness)
        if !ok {
            panic("Item in public parameter tree not of type
witness")

```

```

    }
    witnessTree.ReplaceOrInsert(shr.Witness(index) + witness)
    return true
})
c.ReturnValues.Ascend(func(it btree.Item) bool {
    witness, ok := it.(shr.Witness)
    if !ok {
        panic("Item in return value tree not of type witness")
    }
    witnessTree.ReplaceOrInsert(shr.Witness(index) + witness)
    return true
})

return index, nil
}

```

Status: The Sunspot team has resolved this issue by counting the total number of witnesses in `WitnessLayout()` function in `program.go` using the `CurrentWitnessIndex` of each circuit instead of relying on the opcodes in commit [874279fc80eb06a8d4875c5a6066c55031695c5b](https://github.com/sunspot/abi/commit/874279fc80eb06a8d4875c5a6066c55031695c5b).

SNT-10: `Define()` in `circuit.go` only collects private inputs

Type: High

Files affected: `go/acir/circuit.go`

Description: The `Define()` function for `Circuit` only calls `collectWitnesses()` on `PrivateParameters`. If there are public inputs in the sub-circuit, the compilation will fail since the input list in the `callConnections` won't include the public inputs. For example, the following circuit does not compile:

```

fn main(x: Field, y: pub Field, a: pub u64, b: u64) {
    let z1 = foo(a, y, b);
    let z2 = x + z1 as Field;
    assert(z2 == 5);
}

```

```

}

#[fold]
fn foo(x: u64, z: pub Field, y: u64) -> u64 {
    let r = x & y;
    assert(z == 2);
    r
}

```

Impact: Public inputs in the sub-circuit will result in compilation issues.

Recommendation: Consider using the following function in `Define()` to collect both private and public inputs.

```

func (c *Circuit[T, E]) collectInputWitness(public_tree *btree.BTree,
private_tree *btree.BTree, currentWitnesses
map[shr.Witness]frontend.Variable) []frontend.Variable {
    var vars []frontend.Variable
    num_witness := public_tree.Len() + private_tree.Len()
    for i := range num_witness {
        witness := private_tree.Get(shr.Witness(i))
        if witness == nil {
            witness = public_tree.Get(shr.Witness(i))
        }
        vars = append(vars, currentWitnesses[witness.(shr.Witness)])
    }
    return vars
}

```

Status: The Sunspot team has resolved this issue by collection both private and public inputs in the `collectWitnesses` function in `circuit.go` in commit [6b89bf681274c4c22aa99047fb864c275bc0f722](https://github.com/sunspot/sunspot/commit/6b89bf681274c4c22aa99047fb864c275bc0f722).

SNT-11: `MultiScalarMul.Define` calls `grumpkin.MultiScalarMul` without `algotps.WithCompleteArithmetic()`; performs unsafe arithmetics for scalar multiplication

Type: High

Files affected: `go/acir/black_box_func/multi_scalar_mul.go`

Description: The function `MultiScalarMul.Define` calls `grumpkin.MultiScalarMul` without variadic parameter `opts`. That means the empty `opts` is forwarded to `ScalarMul` in `g1.go`. Based on the scalar `s` being a const or a variable, the `ScalarMul` calls `constScalarMul` or `varScalarMul` respectively with empty `opts`. Calling `algotps.NewConfig(opts...)` with no `opts` essentially takes the default config which has `CompleteArithmetic = false`. Based on `cfg.CompleteArithmetic` the scalar multiplication uses safe and unsafe arithmetic. Since the `CompleteArithmetic` is always false, that results in the function always using unsafe arithmetic such as `AddAssign` and `Double` instead of `AddUnified` which does not handle edge cases such as the point at infinity and $p \neq +-q$.

Impact: A noir circuit using `std::embedded_curve_ops::MultiScalarMul` with edge cases such as zero scalar or point at infinity input can either panic during gnark circuit generation or can potentially generate an underconstrained gnark circuit.

Recommendation: call `grumpkin.MultiScalarMul` with `algotps.WithCompleteArithmetic()` so that safe arithmetic operations are always performed.

Status: This issue has been resolved by calling `grumpkin.MultiScalarMul` with `algotps.WithCompleteArithmetic()` in commit [e29fd6586f9a9f936ace0d71c103f5a4e9d9db76](https://github.com/Conflux-Chain/conflux-go/commit/e29fd6586f9a9f936ace0d71c103f5a4e9d9db76).

SNT-12: Commitment Loop Never Advances index `idx` while creating new proof in recursive aggregation

Type: High

Files affected: `go/acir/black_box_func/recursive_aggregation.go`

Description: The function `newProof` parses a flat `vars` slice that encodes the full Groth16 proof structure in order:

- `vars[0:2]` into `proof.Ar`
- `vars[2:4]` into `proof.Krs`
- `vars[4:8]` into `proof.Bs`
- `vars[8:10]` into `pok`

Followed by `Vars[10:12]`, `Vars[12:14]`,... for commitments. The index `idx` starts at 10 and is intended to advance by 2 for each commitment. But `idx += 2` is missing from the loop body. Every iteration reads `vars[10:12]` i.e., the first commitment's two input variables and assigns it to every slot in the commitments slice. The rest of the commitments `vars` are never read. When `len(commitments) == 1`, the loop runs once and `idx` is never needed again, so the missing increment goes unnoticed but if `len(commitments) > 1` then it results in malformed groth16 proof being passed in recursive aggregation.

Impact: Any noir program that uses recursion to verify groth16 proof will potentially generate a gnark constraint system that encodes a structurally incorrect verification equation which can be exploited by a malicious prover.

Recommendation: Add `idx += 2` at the end of each loop iteration in the commitment loop.

Status: Resolved by correctly updating the `idx`: `idx += 2` in the loop. The fix is added in commit [e29fd6586f9a9f936ace0d71c103f5a4e9d9db76](https://github.com/iden3/noir.js/commit/e29fd6586f9a9f936ace0d71c103f5a4e9d9db76).

SNT-13: `is_infinite` flag for points are ignored

Type: High

Files affected:

- `go/acir/black_box_func/embedded_curve_add.go`
- `go/acir/black_box_func/multi_scalar_mul.go`

Description: In Noir [black_box_solver](#), the `is_infinite` flag in the point has higher priority than the X, Y values and will replace the point with `Affine::zero()`. (The exact behavior in proving depends on the implementation of backends.) In Sunspot, this flag is ignored in `EmbeddedCurveAdd` for `Input1`, `Input2` and `Outputs` and in `MultiScalarMul` for `Points` and `Outputs`.

Impact: Some points might be considered zero in Noir but non-zero in Sunspot, leading to possible attacks where an invalid proof is considered valid in Sunspot. Since the `is_infinite` flags for both input and output are unchecked, provers can set them to any value they want. Anything in the Noir program that branches on the `is_infinite` flag can be controlled by the prover.

Recommendation: Check whether the `is_infinite` flag is set for points and replace the points with zero if it is set.

Status: The Sunspot team has added checks for the `is_infinite` flag in both files in commit [25450efff10848684deaf016658763e60f38403e](#).

SNT-14: `Program.UnmarshalReader` reads `unconstrainedFuncCount` with the wrong endianness and ignores reading any `UnconstrainedFunctions`

Type: High

Files affected: `go/acir/program.go`

Description: The function `Program.UnmarshalReader` reads the unconstrained function count as follows:

```

var unconstrainedFuncCount uint64
    if err := binary.Read(r, binary.BigEndian,
&unconstrainedFuncCount); err != nil {
        if err == io.EOF || err == io.ErrUnexpectedEOF {
            return nil
        }
        return err
    }
}

```

Here, the binary reads uses `binary.BigEndian` order but every other binary read in this codebase uses `binary.LittleEndian`. Moreover, after reading the `unconstrainedFuncCount` the function returns without consuming any Brillig bytecodes. So all Brillig bytecodes remain unread. This results in the `UnconstrainedFunctions` component always being empty in the `Program` struct. The brillig/unconstrained functions execution does not generate any constraint but the output result witnesses are expected to be used in the ACIR circuit and in the proving system. In the absence of `UnconstrainedFunctions`, the deserialized Noir program differs from the original Noir program which can have different witness expectations as the execution output of the unconstrained functions are expected to be used in the noir program and can result in panic during proof generation due to mismatch witnesses.

Impact: Not consuming `UnconstrainedFunctions` in the `Program` result in a different Noir program than expected. Moreover, due to missing `UnconstrainedFunctions`, the expected witness map can also differ from original due to missing unconstrained functions execution outputs and can result in panic during the proof generation.

Recommendation: `unconstrainedFuncCount` must be read in correct endianness(`binary.LittleEndian`) and then brillig bytecode should be read and added to the `Program.UnconstrainedFunctions`. Additionally, the execution result of the unconstrained function should be assigned to the output witnesses without adding any constraints.

Status: The Sunspot team has removed the parsing of the brillig bytecode as it plays no part in the final constraint system in commit

[0cab73d9405fdc15e9314cbe7085e0c6645280a9](https://github.com/sunspot-labs/sunspot/commit/0cab73d9405fdc15e9314cbe7085e0c6645280a9).

SNT-15: Mismatch behavior for `multi_scalar_mul` black box function when `scalar >= Fr`

Type: Medium

Files affected: `go/acir/black_box_func/multi_scalar_mul.go`

Description: In the [noir implementation](#) of `multi_scalar_mul`, it will reject any input `s >= ark_grumpkin::FrConfig::MODULUS`. However, in Sunspot, it will perform `mod(s, Fr)` first. If a user generates a proof with witness `s >= Fr`, it will be accepted in Sunspot but rejected in Noir.

Impact: Same proof might have different behavior in Barretenberg backend and in Sunspot.

Recommendation: Check whether each scalar value is smaller than `Fr` first in `Define()`.

Status: The Sunspot team has resolved this issue by checking whether `scalar >= Fr` for `multi_scalar_mul` in commit [1d1efdb71a5e45a1d03c8c9218c9f53b80195d8a](#).

SNT-16: Incorrect Unmarshalling for `BrilligBytecode`

Type: Medium

Files affected:

- `go/acir/brillig/opcodes/brillig_opcode.go`
- `go/acir/brillig/brillig_bytecode.go`

Description: In the `BrilligBytecode` struct, the `FunctionName` field has been omitted, leading to error during unmarshalling. Similarly, in the `UnmarshalReader()` function for the `BrilligOpcode`, the case for `ACIRBrilligOpcodeReturn` has been skipped, leading to unknown opcode error when the `Return` opcode is encountered.

Impact: Compilation will fail if unconstrained functions are present in the Noir circuit.

Recommendation:

In `brillig_bytecode.go`, add the `FunctionName` field and update the `UnmarshalReader` function:

```
type BrilligBytecode[T shr.ACIRField] struct {
    FunctionName string
    Bytecode     []ops.BrilligOpcode[T]
}

func (b *BrilligBytecode[T]) UnmarshalReader(r io.Reader) error {

    var length uint64
    if err := binary.Read(r, binary.LittleEndian, &length); err != nil
{
        return err
    }

    data := make([]byte, length)
    if _, err := io.ReadFull(r, data); err != nil {
        return err
    }

    b.FunctionName = string(data)

    var bytecodeSize uint64
    if err := binary.Read(r, binary.LittleEndian, &bytecodeSize); err
!= nil {
        return err
    }

    b.Bytecode = make([]ops.BrilligOpcode[T], bytecodeSize)
    for i := uint64(0); i < bytecodeSize; i++ {
        if err := b.Bytecode[i].UnmarshalReader(r); err != nil {
            return err
        }
    }
}
```

```
    return nil
}
```

In `brillig_opcode.go`, add an empty `case ACIRBrilligOpcodeReturn:` in the `UnmarshalReader()` so it won't fall to the default error case.

Status: The Sunspot team has removed the parsing of the brillig bytecode as it plays no part in the final constraint system in commit [0cab73d9405fdc15e9314cbe7085e0c6645280a9](https://github.com/0xcab73d9405fdc15e9314cbe7085e0c6645280a9).

SNT-17: Unsafe operations in ECC arithmetic

Type: Medium

Files affected: `go/sw-grumpkin/g1.go`

Description: The functions `DoubleAndAdd`, `DoubleAndAddSelect`, `AddAssign` and `Double` use `DivUnchecked` with denominators that can be zero. In gnark, `DivUnchecked(a, b)` adds only the constraint $a = b \cdot \text{quotient}$. It does not add any constraint $b \neq 0$. The edge cases such as point at infinity and points `p` and `q` such that $p = +-q$ can have `api.Sub(p1.X, p2.X)` equal to 0 which result in unconstrained output. Due to which there is no constraint forcing the output to lie on the Grumpkin curve or to equal the correct ECC result. These unsafe functions are used in scalar multiplication which is directly reachable from `MultiScalarMul` blackbox function(See [issue 11](#)) can potentially give incorrect results as well as underconstrained circuits for the degenerate cases. For an honest prover with random scalars and generic curve points, the degenerate cases almost never occur in practice. But since there is no check for input points being on the curve and no constraint that inputs are non-degenerate, a malicious prover can deliberately supply such degenerate inputs which can potentially pass the circuit verification resulting in soundness issues.

Impact: Can potentially break the soundness guarantee of any Noir program that uses `multi_scalar_mul` or embedded curve operations over Grumpkin.

Recommendation: The degenerate cases must also be covered in all the functions. Adding `algotps.WithCompleteArithmetic()` to the `grumpkin.MultiScalarMul` in `multi_scalar_mul.go` performs safe operations which also includes degenerate cases. Only safe arithmetic operations should be considered throughout the codebase which also includes degenerate cases such as point at infinity and p, q such that $p = -q$.

Status: This issue has been resolved by calling `grumpkin.MultiScalarMul` with `algotps.WithCompleteArithmetic()` in commit [e29fd6586f9a9f936ace0d71c103f5a4e9d9db76](https://github.com/ethereum/go-ethereum/commit/e29fd6586f9a9f936ace0d71c103f5a4e9d9db76).

SNT-18: Blackbox function `range` in Sunspot does not consider constant witness input

Type: Low

Files affected: `go/acir/black_box_func/range.go`

Description: In the [Noir blackbox solver](#). It will also check the range of a constant input. However in `go/acir/black_box_func/range.go`, it would return early when int input kind is constant. Although in practice it seems less likely that the range function will have a constant input.

Impact: Range function with a constant input will be skipped.

Recommendation: Check the range for both constant and witness input.

Status: The Sunspot team has updated the `Define()` function in `range.go` to check the range for both constant and witness input in commit [60b91eedf40d98807ff2ec8d8cad7d7560d7bfa](https://github.com/ethereum/go-ethereum/commit/60b91eedf40d98807ff2ec8d8cad7d7560d7bfa).

SNT-19: Missing Equality implementation for `BN254Field`

Type: Low

Files affected: `go/bn254/bn254.go`

Description: The `BN254Field.Equals` implements the `ACIRField` interface:

```
func (b BN254Field) Equals(other shr.ACIRField) bool {  
    return true // Implement the equality check logic here  
}
```

Here, the function `Equals` returns true irrespective of the values being compared. It should be noted that the `cmd/root.go` and `cmd/prove.go` both define `type T = *bn254.BN254Field`. This makes `T` a concrete type alias used throughout the entire `cmd` package when calling `acir.LoadACIR[T, E](...)`. All the generic structs such as `Expression[T, E]`, `MulTerm[T, E]` and `LinearCombination[T, E]` get instantiated with `*BN254Field`. When their `Equals()` methods are called, the compiler statically resolves `term.Equals(...)` to `BN254Field.Equals` at compile time.

As a result, all existing ACIR parsing tests that use `Expression.Equals` to verify a deserialized expression against an expected value silently skip the coefficient comparison. If there is any bug in the `UnmarshalReader` that produces wrong BN254 Field Element in mul terms or linear terms that will not be detected. In the current codebase it seems like this only affected the testing and even though there was a flaw in the testing the parsing worked as intended.

Impact: an undetected parsing error can potentially cause the toolchain to compile and prove a different circuit than the one expressed in the Noir circuit.

Recommendation: Implement `Equals` for `BN254Field` and revisit all the tests that directly or indirectly use `BN254Field.Equals`.

Status: The issue has been resolved in commit [87ca2bdfe441ff68ba4140d6c9a29a11d1290e01](https://github.com/noir-lang/noir/pull/1000).

SNT-20: Non-EOF Errors Silently ignored When Reading `numAssertMessages`

Type: Low

Files affected: go/acir/circuit.go

Description: In function `UnmarshalReader`, the `numAssertMessages` is read from the binary as follows:

```
var numAssertMessages uint64
    if err := binary.Read(r, binary.LittleEndian,
&numAssertMessages); err != nil {
        if err == io.EOF {
            c.AssertMessages =
make(map[ops.OpcodeLocation]ap.AssertionPayload[T, E])
            return nil
        }
    }
```

Here, if the binary read fails with any error other than `io.EOF` (e.g. `io.ErrUnexpectedEOF`, see [here](#)), that case is silently ignored. This can potentially result in malformed `Circuit` different from noir circuits.

Impact: Potentially result in malformed circuit different from noir circuit.

Recommendation: Handle the non `io.EOF` error cases as well and return errors for them.

Status: The issue has been resolved in commit [0f07e43ad3481dd7d344906a717923920c09fc2f](#).

SNT-21: errors discarded silently

Type: Low

Files affected:

- go/acir/black_box_func/blake3.go
- go/acir/circuit.go

Description: There are multiple instances in the codebase where the errors are silently discarded.

In file blake3.go, the lines

```
:
hasher.Update(api, *uapi, *uapi64, data) // line 100
:
hasher.Finalize(api, *uapi, output) // line 108
:
h.addChunkChainingValue(api, uapi, [8]uints.U32(chunkCV),
totalChunks) // line 347
:
output.RootOutputBytes(api, uapi, out) // line 371
:
```

discards the errors returned from those functions. Discarding such errors may result in a corrupted or partial hash state in blake3 hash.

In file go/acir/circuit.go, in function `FillWitnessTree`, the line `subCircuit.FillWitnessTree(subCircuitWitnessTree, resolve, index)` returns `(uint32, error)` which is silently discarded. In the same function, in the code block:

```
for _, opcode := range c.Opcodes {
    opcode.FillWitnessTree(witnessTree, index)
}
```

the `opcode.FillWitnessTree` returns a `bool` which is discarded completely.

Impact: Discarding errors can potentially cause the failures to go unnoticed in the codebase and can potentially result in malformed constraints.

Recommendation: Handle all the errors propagation.

Status: The Sunspot team has fixed the error handling for the functions mentioned above in commit [72c5e8c76e5b68893a479ac1656e5a51b6dc54](https://github.com/sunspot/abi/pull/1656).

SNT-22: Wrong logical operator in guard condition while decomposing Scalar

Type: Informational

Files affected: `go/sw-grumpkin/hints.go`

Description: The function `decompose` does sanity check on length of input and output as:

```
if len(inputs) != 1 && len(outputs) != 4 {  
    return errors.New("input/output length mismatch")  
}
```

Here, The condition uses `&&` (AND), meaning the error is only returned when both conditions are simultaneously true but it should return an error if either of the conditions are not met. A single wrong-length argument passes silently. The correct operator is `||` which should be used here. In the current context this is not an issue as gnark's hint infrastructure always calls it with exactly the number of inputs and outputs that were declared in the `api.NewHint` call. Because of that the `if` condition is never touched.

Impact: N/A

Recommendation: Replace `&&` (AND) with `||` (OR)

Status: The Sunspot team has updated the logical operator for the above function in commit [b23c9914dedc651952e1460b9efa632ddd47795e](https://github.com/sunspot/contracts/commit/b23c9914dedc651952e1460b9efa632ddd47795e).

SNT-23: Use `IsWitness()` instead of `FunctionInputKind == 1` to check whether a function input is a witness.

Type: Informational

Files affected: `go/acir/black_box_func/and.go`

Description: In the `FillWitnessTree()` function, it checks the function input type by directly comparing it to `1` instead of using the `IsWitness()` function.

Impact: If the value of the `FunctionInputKind` is updated in the future, the function needs to be updated besides updating the `FunctionInputKind` type.

Recommendation: Check the function input type with the `IsWitness()` function.

Status: The `FillWitnessTree()` function has been removed for the fix of SNT-9.

SNT-24: Use safe type assertion in `Equals()` for `ecdsa_secp256k1.go`

Type: Informational

Files affected: `go/acir/black_box_func/ecdsa_secp256k1.go`

Description: The `other` input for the `Equals` function is using the unchecked version of type assertion, `value := other.(*ECDSA_SECP256K1[T, E])`. Consider using the safe version of type assertion instead.

Impact: The code may panic if the type assertion fails.

Recommendation: Use `value, ok := other.(*ECDSASECP256K1[T, E])` and check the returned `ok` first.

Status: The Sunspot team has updated the function to use safe type assertion in commit [cbda08ae1154ba0bf4464b4598b2572f0c51218a](https://github.com/sunspot/contracts/commit/cbda08ae1154ba0bf4464b4598b2572f0c51218a).

SNT-25: `constantWitnessID` declared in `Expression` is dead code

Type: Informational

Files affected: `go/acir/expression/expression.go`

Description: In struct `Expression`, the `constantWitnessID` is declared but is not used anywhere in the codebase. The comment mentions `constantWitnessID` as a constant term in the expression but there is a field named `Constant` that is used as a constant in the entire codebase. In Noir's ACIR, the constant in an expression is always a literal (not a witness reference), so `constantWitnessID` serves no purpose in the current design. The field is dead code and should be removed to avoid any confusion in the codebase.

Impact: N/A

Recommendation: Removed `constantWitnessID` as it is a dead code.

Status: The issue has been resolved in commit [73e46a7180969d8ef408d0f78c0d6e3399f0ddad](https://github.com/sunspot/contracts/commit/73e46a7180969d8ef408d0f78c0d6e3399f0ddad).

SNT-26: Test cases does not handle all cases in testing

Type: Informational (out of scope)

Files affected: Across the codebase

Description: The testing in the codebase mainly covers two things: deserialization tests and End-to-end honest prover test. there are no tests that attempt to forge an output and expect the circuit to reject it. The entire test suite only exercises the honest prover path. Moreover, Most testgen generators generate no fixtures at all for multiple files including `multi_scalar_mul.rs`, `embedded_curve_add.rs`, `ecdsa_secp256k1.rs`, `ecdsa_secp256r1.rs`, `xor.rs`, `recursive_aggregation.rs`. The generators that do exist just cover 1-2 instances that too with very small values that may not trigger any edge cases issues. For example, the AND/XOR should generate testgen for different nBits including nBits > 64. For elliptic curves, testing edge cases such as ecc operation on invalid points, infinity point, zero scalar, scalar larger than field size etc should be covered to ensure that the circuit behavior is as expected. In addition to that, for generating ACIR field testgen or Expressions, none of the `FieldElement` values are greater than a u32. The actual circuits will have values much higher than u32 (arbitrary 254-bit field element). Not testing such cases may have issues that can silently go undetected.

Impact: Edge cases issues may go undetected

Recommendation: More aggressive testing is needed which covers all possible cases including edge cases behaviour. That not only ensures correctness but also shows the expected behavior of the circuit through testing.

Status: N/A

SNT-27: out-of-bounds dynamic-array write silently becomes a no-op

Type: High

Files affected: `go/acir/memory_op/memory_op.go`

Description: In Noir, `arr[idx] = val` requires `idx` to be a valid index of `arr` for a fixed length array. The out-of-range index makes the constraint system unsatisfiable but Sunspot's translation of the Acir write memory opcode does not enforce this. In the `Define` function

```

if o.IsWrite {
    newTable := logderivlookup.New(api)
    tableLen := (*table).Insert(0)
    for i := 0; i < tableLen; i++ {
        isWritable := api.IsZero(api.Sub(indexVar,
frontend.Variable(i)))
        updated := api.Select(isWritable, valueVar,
(*table).Lookup(i)[0])
        newTable.Insert(updated)
    }
    o.Memory[o.BlockID] = &newTable
    return nil
} else {
    api.AssertIsEqual((*table).Lookup(indexVar)[0], valueVar)
    return nil
}

```

For each of the `tableLen` constant positions `i`, the code computes a one-hot indicator `isWritable_i = (idx == i)` and uses it to select either the new value or the position's existing value. This works correctly when `idx` is actually within `[0, tableLen)`, ensuring that exactly one position is updated. However, neither this loop nor any other part of `Define` constrains the `isWritable_i` indicators to sum to 1. If `idx` is outside `[0, tableLen)`, all `isWritable_i` values are necessarily 0, causing every `Select` to retain the corresponding old value. As a result, `newTable` becomes bit-for-bit identical to the original table, yielding a witness assignment that satisfies all constraints in the loop for any out-of-range `idx`.

For the noir circuit:

```

fn main(idx: u32, val: Field) -> pub [Field; 4] {
    let mut arr = [10, 20, 30, 40];
    arr[idx] = val;
    arr
}

```

In Sunspot, the tampered witness proves the public output `[10, 20, 30, 40]` for `idx=4`, `val=99` but in reality, `idx=4` is out of bounds. The honest program has no

defined output for it at all, and Noir's own toolchain (and bb) correctly refuse to produce anything. Sunspot instead treats it as a no-op and happily proves it.

Impact: Any Noir program that performs a dynamic-index write into an array or Vec-backed block can be incorrectly proved for an out-of-bound index.

Recommendation: After the loop in the `IsWrite` branch, accumulate `sum := api.Add(sum, isWritable_i)` across all `i`, then `api.AssertIsEqual(sum, 1)`, forcing exactly one table position to match the write index, mirroring the implicit guarantee the READ branch already gets for free from `logderivlookup.Lookup`

Status: Fixed in the PR <https://github.com/reilabs/sunspot/pull/57>.

SNT-28: pub-declared Noir return values are silently compiled as Secret witnesses

Type: Critical

Files affected: go/acir/acir.go

Description: Noir's ACIR bytecode tracks a function's witnesses as three disjoint sets: `private_parameters`, `public_parameters`, and `return_values`. A pub-declared return value's visibility is not part of `public_parameters` but recorded separately, in the ABI's `return_type.visibility` field. Sunspot's ACIR to gnark compiler, `*ACIR[T, E]).Compile()` decides which R1CS witnesses become public Groth16 variables with exactly one loop:

```
for index, param := range a.ABI.Params() {
    if param.Visibility ==
        hdr.ACIRParameterVisibilityPublic {
        witnessMap[shr.Witness(uint32(index)+mainStart)] =
            builder.PublicVariable(...)
    }
}
for i := uint32(0); i < totalSlots; i++ {
    if _, ok := witnessMap[shr.Witness(i)]; !ok {
```

```

        witnessMap[shr.Witness(i)] =
        builder.SecretVariable(...)
    }
}

```

`a.ABI.Params()` flattens only `a.Parameters` (function arguments). The `ACIRABI` struct does have a `ReturnType` `*ACIRReturnType` field, correctly parsed from the ACIR JSON's `return_type` object including its Visibility, but that field is never read anywhere else in the codebase. So every return-value witness regardless of being declared `pub` falls through to the second loop and is allocated `builder.SecretVariable(...)`.

For circuit:

```

fn main(x: Field) -> pub Field {
    x + 1
}

```

A proof generated for `x = 7` and output 8 is indistinguishable from one generated for `x = 99` and output 100: both verify against the same verification key with the same empty public witness. The `.pw` file is identical in both cases (12 zero bytes), so the proof is not bound to the claimed output. The Noir's bb correctly ties the claimed output to the proof whereas Sunspot does not.

Impact: This affects every Sunspot-compiled circuit whose public statement is expressed via a `pub` return value. For any such circuit, The Groth16 proof carries zero public inputs tied to the return value. A prover can produce a valid-looking proof for any output value consistent with any choice of the inputs, and no verifier has anything to check the claimed output against.

Recommendation: Update `Compile()` so that `pub` return values are treated as public inputs, just like `pub` function parameters. Use the return-value witness indices from the ACIR and mark them as `PublicVariable` instead of `SecretVariable`. Also, make sure all public witnesses are created before the private ones, as required by gnark.

Status: Fixed in PR <https://github.com/reilabs/sunspot/pull/57>.

References

- <https://github.com/noir-lang/noir/tree/v1.0.0-beta.18>
- <https://docs.gnark.consensus.net/overview>
- <https://noir-lang.org/docs/>
- <https://github.com/Consensus/gnark>
- github.com/consensus/gnark-crypto